

Trabajo Práctico N°6 - Base de Datos

Normalización BCNF y Desnormalización Controlada

Proyecto: “Food Store”

Repositorio: https://github.com/FacundoDeseff/Tps_BasedeDatos

**Facundo Deseff
Facundo Ramírez
Comisión 4**

Normalización en Forma Normal de Boyce-Codd (FNBC)

Dependencias Funcionales (punto 4.2.a)

De la regla de negocio se desprenden, en notación formal:

F1: {LoteID, DepositoID} → ResponsableControlID (Para un lote y un depósito interviniente dados, el responsable queda unívocamente determinado.)

F2: ResponsableControlID → DepositoID (Cada responsable pertenece a un único depósito como dato maestro.)

Clausura y claves candidatas (punto 4.2.b)

1. Atributos individuales:

- **LoteID**⁺ = {LoteID}
- **DepositoID**⁺ = {DepositoID}
- **ResponsableControlID**⁺ = {ResponsableControlID, DepositoID} (por la regla F2)

2. Combinaciones de atributos:

- **{LoteID, DepositoID}**⁺ = {LoteID, DepositoID, ResponsableControlID} (por la regla F1)

Determina a toda la tabla de forma mínima → **Clave Candidata 1.**

- **{LoteID, ResponsableControlID}**⁺ = {LoteID, ResponsableControlID, DepositoID} (ya que ResponsableControlID determina DepositoID por F2)

Determina a toda la tabla de forma mínima → **Clave Candidata 2.**

- **{DepositoID, ResponsableControlID}**⁺ = {DepositoID, ResponsableControlID}

No logra determinar a LoteID, por lo que **no es clave.**

Resultados del análisis:

- **Claves candidatas:** (LoteID, DepositoID) y (LoteID, ResponsableControlID).
- **Atributos primos:** LoteID, DepositoID y ResponsableControlID (todos forman parte de al menos una clave candidata).
- **Atributos no primos:** Ninguno.
- **Clave primaria seleccionada:** (LoteID, DepositoID) (manteniendo la definición original).

Violación FNBC (punto 4.2.c)

Definición formal FNBC: toda DF no-trivial $X \rightarrow Y$ exige que X sea superclave. F2: ResponsableControlID $\rightarrow$ DepositoID es no-trivial y ResponsableControlID+ $\neq$ {LoteID, DepositoID, ResponsableControlID}, luego su determinante no es superclave. Viola FNBC. F1 no viola porque {LoteID, DepositoID} sí es superclave

Anomalías con la instancia (501,30,801),(502,30,801),(503,31,802) (punto 4.2.d)

Insertión: no se puede registrar que un nuevo responsable 803 pertenece al depósito 32 sin inventar un lote_id ficticio, porque la PK (lote_id, deposito_id) obliga a informar lote. Borrado: si se elimina la única fila (503,31,802) se pierde el dato maestro 802 $\rightarrow$ 31. Actualización: si 801 cambia del depósito 30 al 33 hay que actualizar 2 filas (501 y 502); si falla una queda 801 $\rightarrow$ 30 y 801 $\rightarrow$ 33 a la vez.

Descomposición BCNF (punto 4.2.e)

Se descompuso la tabla original en dos nuevas relaciones:

1. responsable_deposito:
 - Clave Primaria: responsable_control_id
 - Atributos: deposito_id
2. control_lote:
 - Clave Primaria Compuesta: (lote_id, responsable_control_id)
 - Clave Foránea: responsable_control_id $\rightarrow$ responsable_deposito

Vista de Compatibilidad (punto 4.2.e)

Para mantener la compatibilidad con consultas existentes se creó la vista:

```
CREATE OR REPLACE VIEW v_control_lote_almacen AS
```

```
SELECT
```

```
    cl.lote_id,
```

```
    rd.deposito_id,
```

```
    cl.responsable_control_id
```

```
FROM control_lote cl
```

```
JOIN responsable_deposito rd
```

```
ON rd.responsable_control_id = cl.responsable_control_id;
```

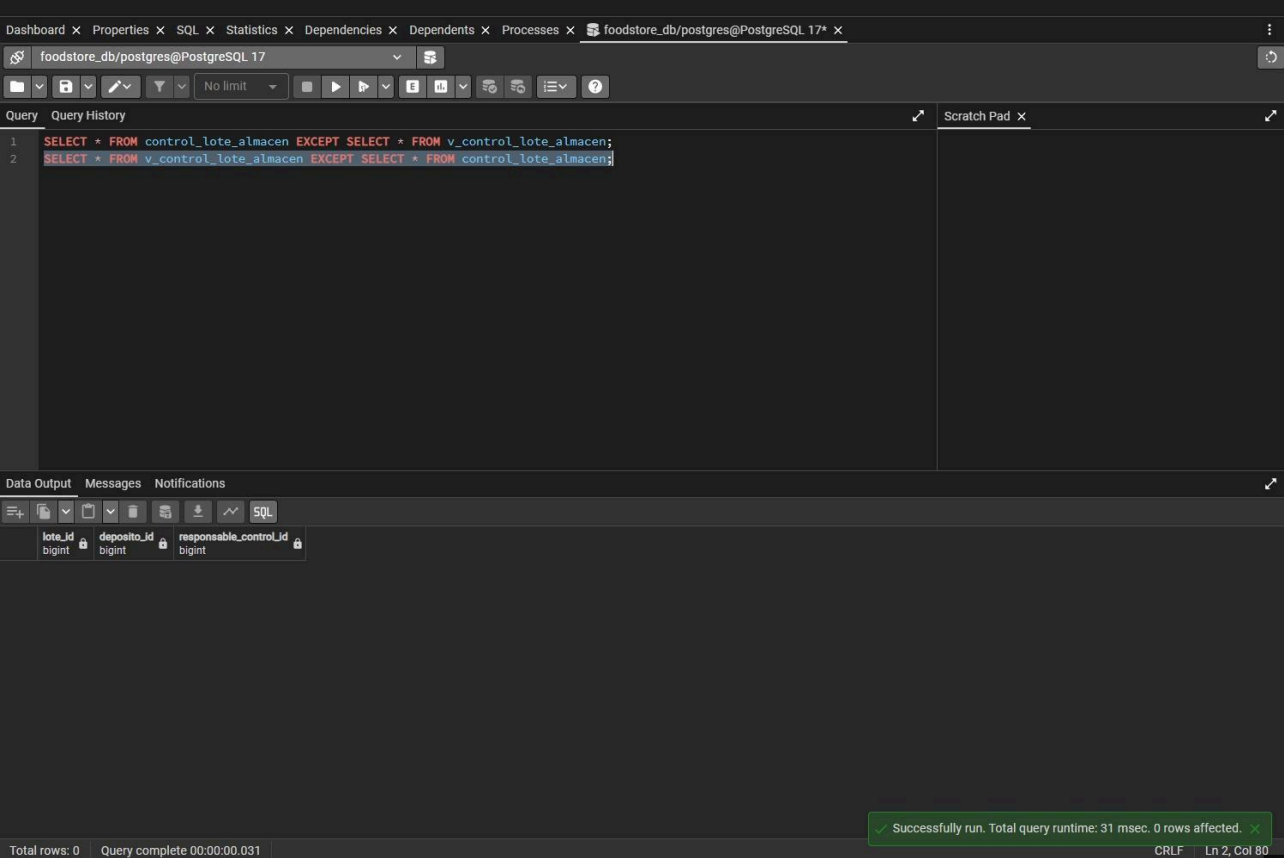
Verificación de Pérdida de Datos (punto 4.2.f)

Justificación teórica sin-pérdida: $R1(\text{responsable_deposito}) \cap R2(\text{control_lote}) = \{\text{ResponsableControlID}\}$, que es PK (superclave) en R1. Por teorema de descomposición binaria el JOIN es sin pérdida.

Se ejecutaron consultas de diferencia de conjuntos (EXCEPT) entre la tabla original y la vista reconstruida:

```
SELECT * FROM control_lote_almacen
EXCEPT
SELECT * FROM v_control_lote_almacen;
SELECT * FROM v_control_lote_almacen
EXCEPT
SELECT * FROM control_lote_almacen;
```

Resultado: Se obtuvieron 0 filas en ambas consultas, lo que garantiza formalmente una descomposición sin pérdida de datos (lossless-join decomposition).



The screenshot shows a PostgreSQL IDE interface with the following components:

- Top Bar:** Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, foodstore_db/postgres@PostgreSQL 17*.
- Query Editor:** Contains two SQL queries:

```
1 SELECT * FROM control_lote_almacen EXCEPT SELECT * FROM v_control_lote_almacen;
2 SELECT * FROM v_control_lote_almacen EXCEPT SELECT * FROM control_lote_almacen;
```
- Data Output:** Shows the results of the queries, which are empty. The columns are: lote_id (bigint), deposito_id (bigint), responsable_control_id (bigint).
- Status Bar:** Total rows: 0, Query complete 00:00:00.031, Successfully run. Total query runtime: 31 msec. 0 rows affected.

Desnormalización Controlada para Optimización

Justificación del patrón elegido (punto 5.2.b)

Se elige Vista Materializada porque el EXPLAIN ANALYZE antes muestra Sequential Scan + Hash Join + HashAggregate sobre cuatro tablas (detalle_pedido, producto, categoria, pedido) ejecutado muchas veces por minuto para el Top 5 diario; la sincronía se garantiza con REFRESH MATERIALIZED VIEW CONCURRENTLY diario vía pg_cron/cron al cierre de jornada; es reversible sin pérdida con DROP MATERIALIZED VIEW porque la fuente de verdad OLTP queda intacta.

Solución Implementada

Se optó por una Vista Materializada (mv_top_categorias_diarias) pre-calculando las ventas agregadas por categoría y fecha, complementada con un índice en la columna de filtrado temporal:

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_top_categorias_diarias AS SELECT  
c.nombre AS categoria, SUM(dp.subtotal) AS total_vendido, ped.fecha
```

```
FROM detalle_pedido dp
```

```
JOIN producto pr ON pr.id_producto = dp.producto_id
```

```
JOIN categoria c ON c.id_categoria = pr.id_categoria
```

```
JOIN pedido ped ON ped.id_pedido = dp.pedido_id
```

```
WHERE dp.eliminado = FALSE AND ped.eliminado = FALSE
```

```
GROUP BY c.nombre, ped.fecha;
```

```
CREATE INDEX IF NOT EXISTS idx_mv_top_cat_fecha ON mv_top_categorias_diarias(fecha);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_mv_top_cat ON  
mv_top_categorias_diarias(categoria, fecha);
```

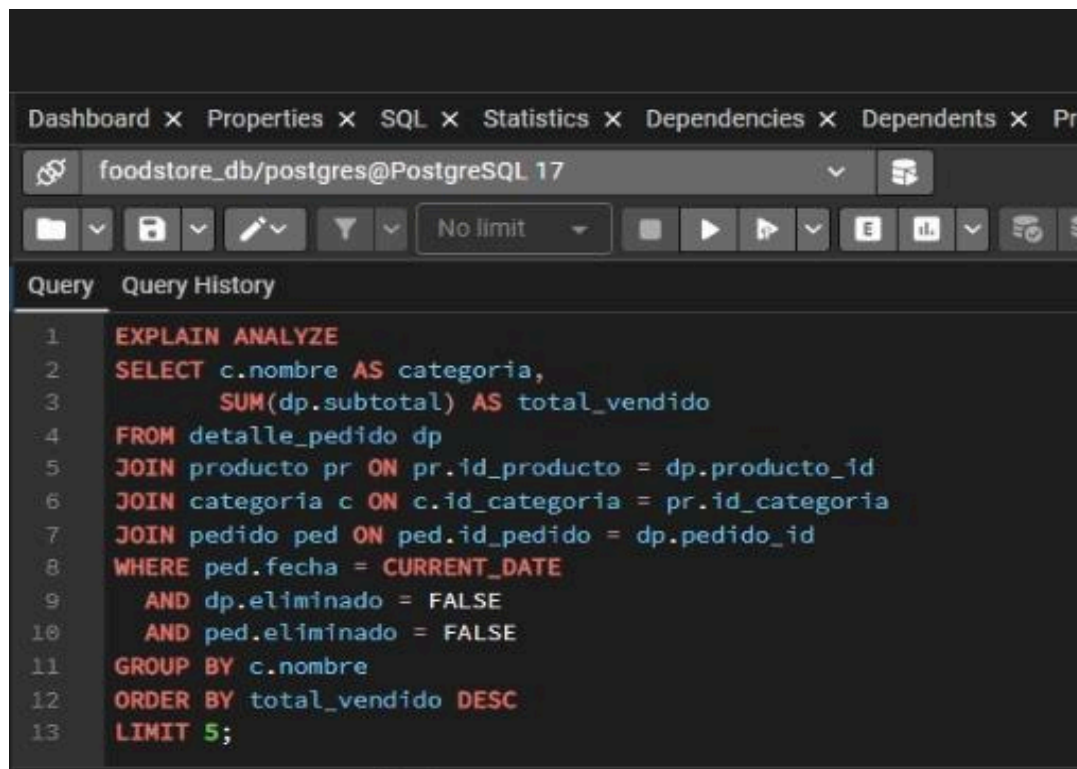
Comparativa de Rendimiento (EXPLAIN ANALYZE) (punto 5.2.a y 5.2.d)

Mediciones sobre instancia poblada (~200k pedidos, ~117k líneas de detalle_pedido):

Estrategia	Tiempo de Ejecución	Tipo de Acceso
Consulta Original (3FN)	77.641 ms	GroupAggregate + Parallel Hash Join + Parallel Seq Scan
Vista Materializada + Índice	0.019 ms	Index Scan (idx_mv_top_cat_fecha)

Mejora lograda: Reducción drástica del tiempo de respuesta (más del 99% de optimización), pasando de procesar un JOIN complejo a una lectura indexada directa.

Nota (opción documentada): la consulta filtra por CURRENT_DATE sin ventas del día (0 filas devueltas, ~200k pedidos filtrados). El costo medido corresponde al recorrido y JOIN sobre la instancia poblada, válido como evidencia del antes.



```
1  EXPLAIN ANALYZE
2  SELECT c.nombre AS categoria,
3         SUM(dp.subtotal) AS total_vendido
4  FROM detalle_pedido dp
5  JOIN producto pr ON pr.id_producto = dp.producto_id
6  JOIN categoria c ON c.id_categoria = pr.id_categoria
7  JOIN pedido ped ON ped.id_pedido = dp.pedido_id
8  WHERE ped.fecha = CURRENT_DATE
9         AND dp.eliminado = FALSE
10        AND ped.eliminado = FALSE
11  GROUP BY c.nombre
12  ORDER BY total_vendido DESC
13  LIMIT 5;
```

Data Output		Messages	Notifications
≡+ ▼ ▼ ▼ ▼ ▼ ▼ ▼ SQL			
	QUERY PLAN text 🔒		
1	Limit (cost=8871.76..8871.76 rows=1 width=40) (actual time=71.036..77.555 rows=0 loops=1)		
2	-> Sort (cost=8871.76..8871.76 rows=1 width=40) (actual time=71.035..77.553 rows=0 loops=1)		
3	Sort Key: (sum(dp.subtotal)) DESC		
4	Sort Method: quicksort Memory: 25kB		
5	-> GroupAggregate (cost=5707.14..8871.75 rows=1 width=40) (actual time=71.024..77.542 rows=0 loops=1)		
6	Group Key: c.nombre		
7	-> Nested Loop (cost=5707.14..8871.73 rows=1 width=14) (actual time=71.023..77.540 rows=0 loops=1)		
8	Join Filter: (c.id_categoria = pr.id_categoria)		
9	-> Index Scan using categoria_nombre_key on categoria c (cost=0.13..12.21 rows=5 width=16) (actual time=0.005..0.013 rows=...		
10	-> Materialize (cost=5707.01..8859.45 rows=1 width=14) (actual time=14.201..15.505 rows=0 loops=5)		
11	-> Nested Loop (cost=5707.01..8859.44 rows=1 width=14) (actual time=71.002..77.518 rows=0 loops=1)		
12	-> Gather (cost=5706.72..8859.12 rows=1 width=14) (actual time=71.001..77.514 rows=0 loops=1)		
13	Workers Planned: 1		
14	Workers Launched: 1		
15	-> Parallel Hash Join (cost=4706.72..7859.02 rows=1 width=14) (actual time=4.397..4.399 rows=0 loops=2)		
16	Hash Cond: (dp.pedido_id = ped.id_pedido)		
17	-> Parallel Seq Scan on detalle_pedido dp (cost=0.00..2843.47 rows=117647 width=22) (never executed)		
18	Filter: (NOT eliminado)		
19	-> Parallel Hash (cost=4706.71..4706.71 rows=1 width=8) (actual time=4.259..4.259 rows=0 loops=2)		
20	Buckets: 1024 Batches: 1 Memory Usage: 0kB		
21	-> Parallel Seq Scan on pedido ped (cost=0.00..4706.71 rows=1 width=8) (actual time=8.483..8.483 rows=0 loops=1)		
22	Filter: ((NOT eliminado) AND (fecha = CURRENT_DATE))		
23	Rows Removed by Filter: 200000		
24	-> Index Scan using producto_pkey on producto pr (cost=0.29..0.32 rows=1 width=16) (never executed)		
25	Index Cond: (id_producto = dp.producto_id)		
26	Planning Time: 0.492 ms		
27	Execution Time: 77.641 ms		

Después:

The screenshot shows a PostgreSQL query editor interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, and Processes. The main query editor displays the following SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT categoria, total_vendido
3 FROM mv_top_categorias_diarias
4 WHERE fecha = CURRENT_DATE
5 ORDER BY total_vendido DESC
6 LIMIT 5;
```

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Limit	8.45..8.46	1	13	0.008..0.009	0	1
2	Sort	8.45..8.46	1	13	0.008..0.008	0	1
3	Sort Key						
4	Sort Method	quicksort		Memory: 25kB			
5	Index Scan	0.42..8.44	1	13	0.004..0.004	0	1
6	Index Cond						
7	Planning Time				1.070 ms		
8	Execution Time				0.019 ms		

Estrategia de Sincronización y Auditoría (puntos 5.2.c y 5.2.e)

Dado que las Vistas Materializadas no se actualizan automáticamente en PostgreSQL, se refresca con:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_categorias_diarias; --
Automático diario 23hs con pg_cron: -- SELECT cron.schedule('refresh_top_cat','0 23 * *
*','REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_categorias_diarias');
```

Para verificar que la vista desnormalizada no presente diferencias con las tablas OLTP transaccionales, se ejecutó la siguiente consulta de auditoría:

```
SELECT c.nombre AS categoria, SUM(dp.subtotal) AS total_calculado, mv.total_vendido AS total_registrado
```

```
FROM detalle_pedido dp
```

```
JOIN producto pr ON pr.id_producto = dp.producto_id
```

```
JOIN categoria c ON c.id_categoria = pr.id_categoria
```

```
JOIN pedido ped ON ped.id_pedido = dp.pedido_id LEFT
```

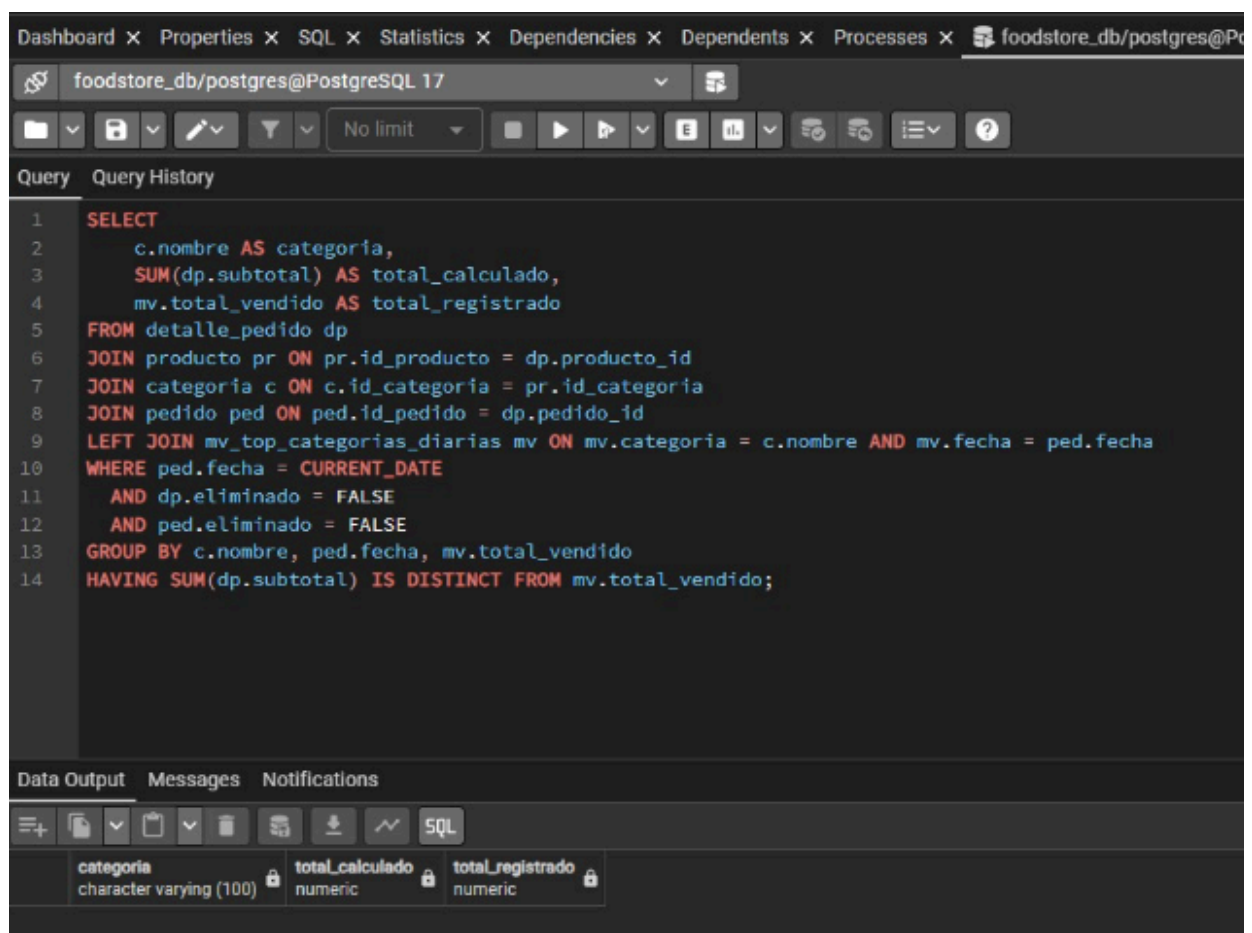
```
JOIN mv_top_categorias_diarias mv ON mv.categoria = c.nombre AND mv.fecha = ped.fecha
```

```
WHERE ped.fecha = CURRENT_DATE AND dp.eliminado = FALSE AND ped.eliminado = FALSE
```

```
GROUP BY c.nombre, ped.fecha, mv.total_vendido HAVING SUM(dp.subtotal) IS DISTINCT
```

```
FROM mv.total_vendido;
```

Resultado: 0 filas devueltas, confirmando la consistencia total de saldos entre el modelo normalizado y el desnormalizado.



The screenshot shows a PostgreSQL query editor interface. The top bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, and Processes. The main window displays the following SQL query:

```
1 SELECT
2     c.nombre AS categoria,
3     SUM(dp.subtotal) AS total_calculado,
4     mv.total_vendido AS total_registrado
5 FROM detalle_pedido dp
6 JOIN producto pr ON pr.id_producto = dp.producto_id
7 JOIN categoria c ON c.id_categoria = pr.id_categoria
8 JOIN pedido ped ON ped.id_pedido = dp.pedido_id
9 LEFT JOIN mv_top_categorias_diarias mv ON mv.categoria = c.nombre AND mv.fecha = ped.fecha
10 WHERE ped.fecha = CURRENT_DATE
11        AND dp.eliminado = FALSE
12        AND ped.eliminado = FALSE
13 GROUP BY c.nombre, ped.fecha, mv.total_vendido
14 HAVING SUM(dp.subtotal) IS DISTINCT FROM mv.total_vendido;
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The output is a table with three columns: 'categoria' (character varying (100)), 'total_calculado' (numeric), and 'total_registrado' (numeric). The table is currently empty, indicating that no rows were returned by the query.

Conclusión

En este trabajo práctico contrastamos dos enfoques clave del diseño de bases de datos: la normalización y la desnormalización.

Por un lado, la normalización a Forma Normal de Boyce-Codd (FNBC) nos permitió eliminar anomalías de inserción, modificación y borrado, asegurando la integridad teórica del modelo mediante una descomposición sin pérdida de datos.

Por otro lado, comprobamos que en consultas analíticas la normalización extrema penaliza el rendimiento. Para resolverlo, implementamos una vista materializada que redujo el tiempo de respuesta de ~77 ms a 0.019 ms (más de un 99% de mejora), manteniendo la consistencia del sistema mediante un esquema de refresco automatizado y consultas de auditoría.

En conclusión, un diseño sólido requiere equilibrar la rigidez teórica de la normalización con las necesidades reales de rendimiento del negocio.